

Trabajo practico final de Aprendizaje por Refuerzo II

Por Lautaro Medina.

Introducción:

En este trabajo se desarrolló un agente basado en aprendizaje por refuerzo para resolver el entorno MountainCarContinuous-v0 de Gymnasium. El objetivo consiste en aprender una política que permita controlar la aceleración de un vehículo para alcanzar la cima de una montaña de forma autónoma.

Para la implementación se utilizó el algoritmo Soft Actor-Critic (SAC) mediante la biblioteca Stable-Baselines3. Además del entrenamiento del agente, se incorporaron mecanismos de evaluación automática, almacenamiento del mejor modelo y generación de métricas que permiten analizar la evolución del aprendizaje y el desempeño obtenido durante el entrenamiento.

Descripción del problema:

El entorno MountainCarContinuous-v0 representa un vehículo ubicado en un valle que debe alcanzar la cima de una montaña. Sin embargo, la potencia del motor no es suficiente para subir la pendiente directamente, por lo que el agente debe aprender a generar impulso desplazándose hacia ambos lados antes de intentar llegar al objetivo.

El estado del entorno está compuesto por dos variables: la posición y la velocidad del vehículo. A partir de esta información, el agente debe decidir una acción continua que representa la aceleración aplicada en cada instante. Durante el entrenamiento recibe una recompensa en función del desempeño obtenido, incentivando aquellas acciones que permiten alcanzar la meta de manera eficiente y penalizando el uso excesivo de aceleración.

Algoritmo SAC:

Para resolver este problema se seleccionó el algoritmo Soft Actor-Critic (SAC) debido a que MountainCarContinuous-v0 posee un espacio de acciones continuas.

SAC fue desarrollado específicamente para problemas de control continuo y combinando una arquitectura Actor-Critic con un mecanismo de exploración basado en entropía favorece la búsqueda de políticas más robustas y reduce la probabilidad de converger prematuramente a soluciones de baja calidad.

Se considero además las limitaciones locales de computo tanto en la selección del algoritmo como en la del problema.

SAC utiliza un Replay Buffer, permitiendo reutilizar experiencias obtenidas durante el entrenamiento. Esto mejora la eficiencia del aprendizaje, ya que el agente requiere menos interacciones con el entorno para alcanzar un buen desempeño en comparación con otros

algoritmos. En este trabajo, el entrenamiento finalizó automáticamente luego de aproximadamente 590.000 timesteps, obteniendo una recompensa promedio de evaluación de 94.8, lo que demuestra que SAC resultó una alternativa adecuada para resolver este problema.

Implementacion:

La implementación se realizó en Python utilizando Gymnasium para el entorno y Stable-Baselines3 para el agente SAC. También se utilizó PyTorch con CUDA, permitiendo ejecutar el entrenamiento sobre GPU cuando se encontraba disponible.

El script crea el entorno MountainCarContinuous-v0, lo envuelve con Monitor para registrar métricas por episodio y luego instancia el modelo SAC con una política MlpPolicy. Se configuró un *Replay Buffer*, una tasa de aprendizaje de 0.0003, un tamaño de batch de 256 y una arquitectura de red neuronal de dos capas de 256 neuronas.

Además del entrenamiento principal, se incorporó:

- CheckpointCallback: guarda versiones intermedias del modelo.
- EvalCallback: evalúa periódicamente el desempeño del agente.
- StopTrainingOnNoModelImprovement: detiene el entrenamiento si el modelo deja de mejorar.

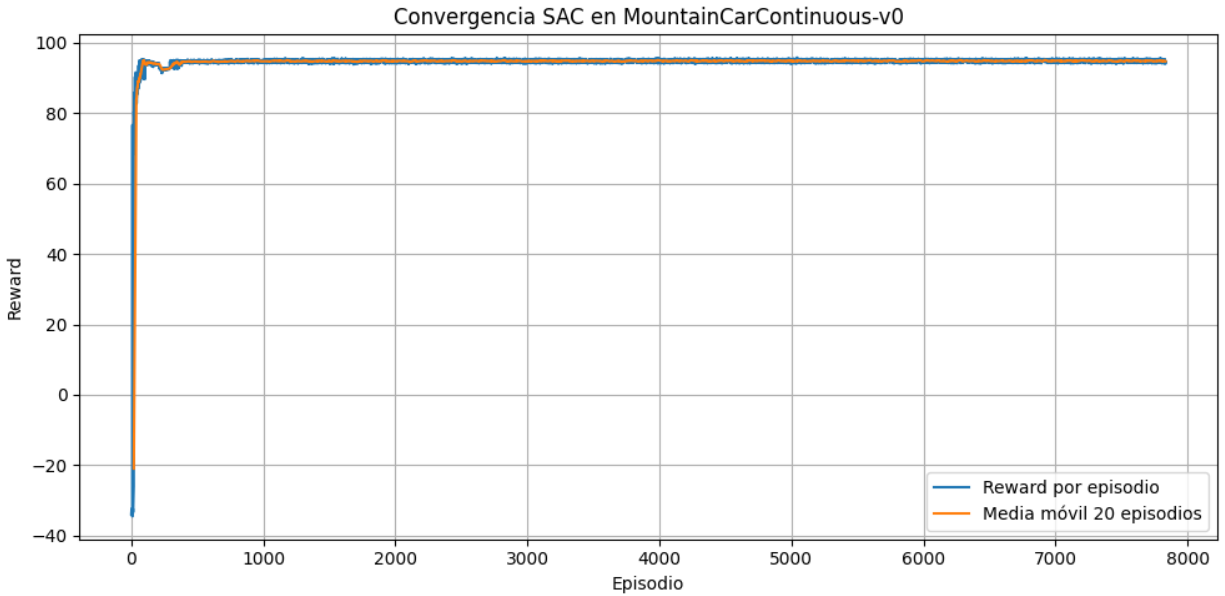
Con esto se puede ahorrar tiempo en el caso de que el modelo converja en una solución antes de alcanzar el límite de timesteps (1.000.000 en este caso).

También se registraron las recompensas y longitudes de cada episodio, generando automáticamente los gráficos utilizados para analizar la convergencia y el desempeño final del agente.

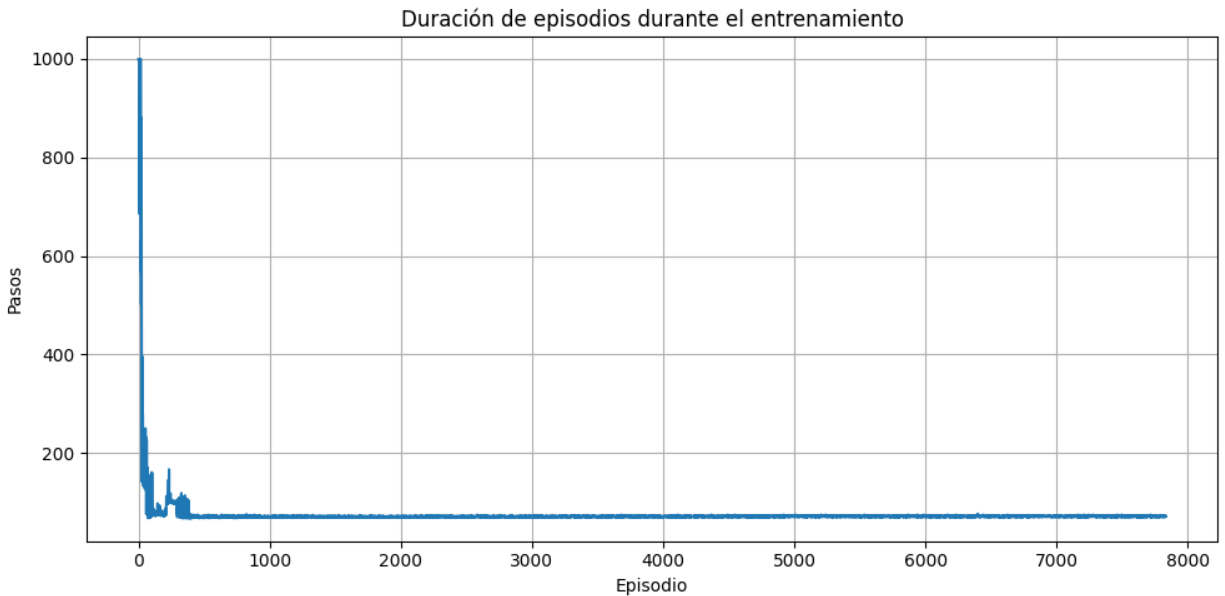
Resultados:

5. Resultados

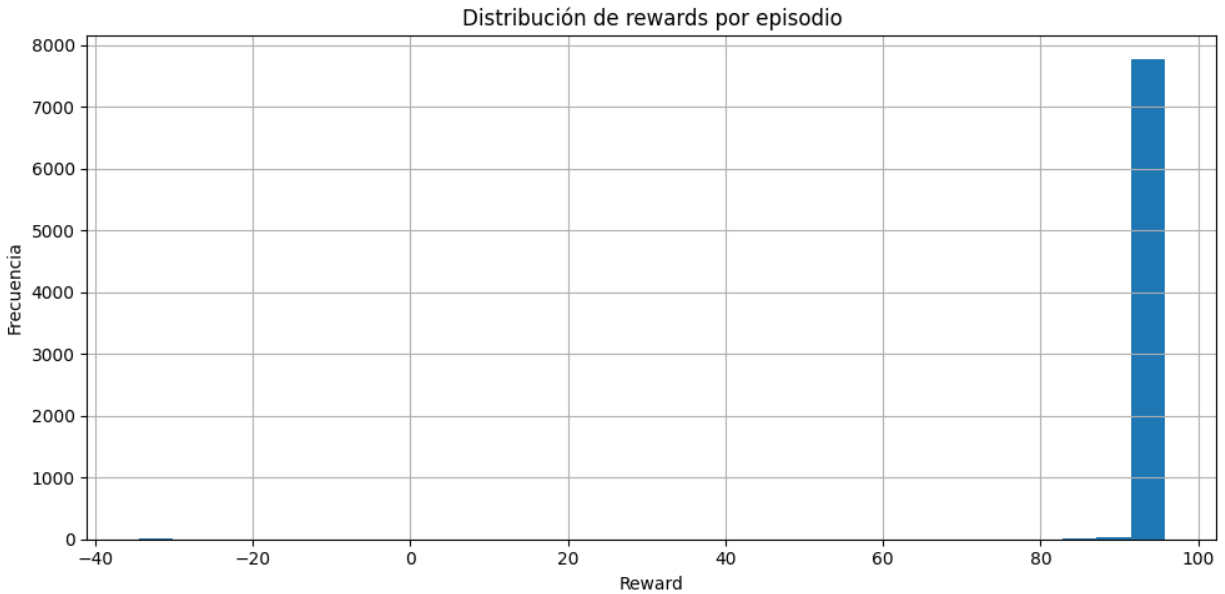
Durante el entrenamiento el agente alcanzó una recompensa promedio de evaluación de 94.8, finalizando automáticamente luego de 590.000 timesteps mediante el criterio de parada temprana implementado. Esto indica que el algoritmo dejó de presentar mejoras significativas y que la política aprendida logró resolver el entorno de forma consistente.



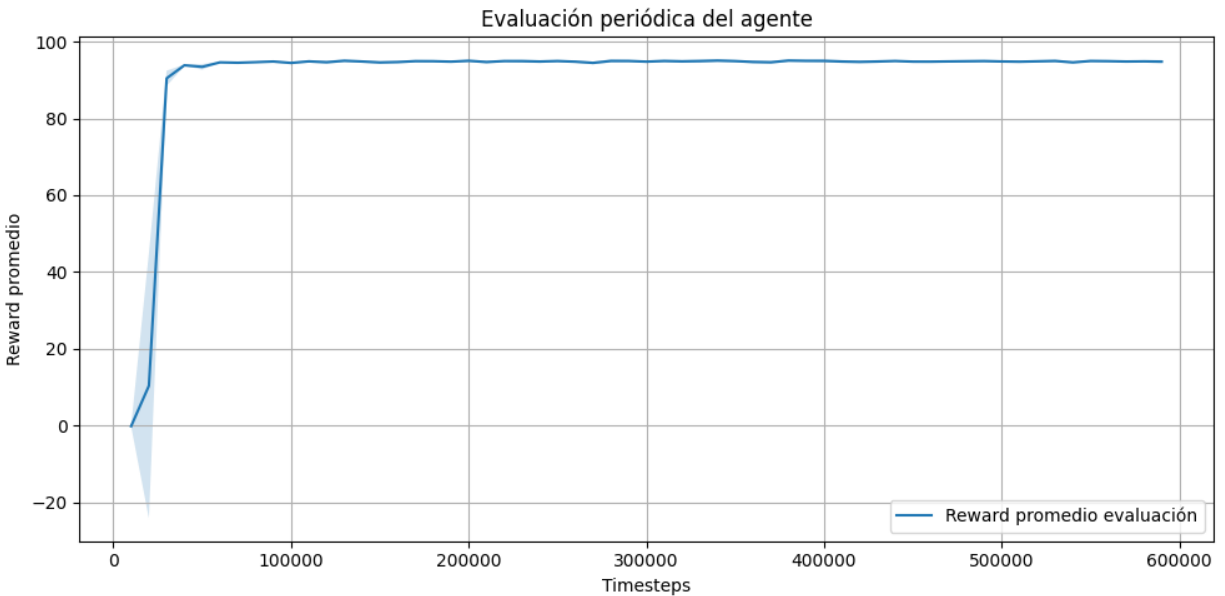
La Figura 1 presenta la evolución de la recompensa obtenida por episodio durante el entrenamiento. Se puede ver que el modelo rápidamente llegó al límite que obtendría de recompensas, teniendo algunas variaciones mas grandes en los primeros 500 episodios pero luego solo mantuvo suficientes variaciones como para evitar el Earlystopping.



La Figura 2 muestra la longitud de los episodios registrados durante el entrenamiento. Inicialmente el agente requiere una mayor cantidad de pasos para completar cada episodio debido a la exploración del entorno. La política mejora rápidamente durante los primeros 500 episodios, coincidiendo esto naturalmente con lo presentado en el grafico de recompensas.



En la Figura 3 se presenta la distribución de las recompensas obtenidas. La mayor concentración de episodios se encuentra en valores elevados de recompensa, lo que demuestra que el agente aprendió rápidamente y tuvo pocas variaciones en valores inferiores de recompensa.



Finalmente, la Figura 5 presenta la recompensa promedio obtenida durante las evaluaciones periódicas realizadas sobre el mejor modelo. A diferencia de la recompensa por episodio, estas evaluaciones se realizan sin exploración adicional, por lo que representan una medida más confiable del desempeño real del agente. Considerando las

pocas diferencias a gran escala, este grafico si bien se considero en un principio como algo pertinente termina dando información muy similar a gráficos previos de recompensa.

Conclusión:

En conjunto, los resultados obtenidos muestran que el algoritmo SAC fue capaz de aprender una política adecuada para resolver el entorno MountainCarContinuous-v0. La convergencia observada en las distintas métricas y la estabilidad alcanzada durante las evaluaciones indican que el agente aprendió una estrategia efectiva para controlar la aceleración del vehículo y alcanzar la cima de la montaña de forma consistente.

Idealmente se hubiera probado otros algoritmos para comparar pero las limitaciones de tiempo y de computa dejaron este análisis limitado a SAC.